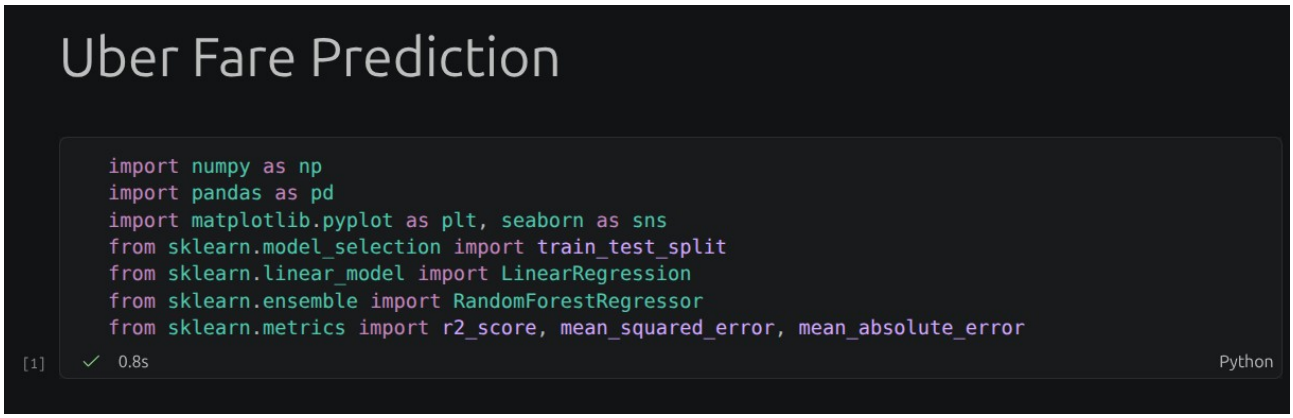
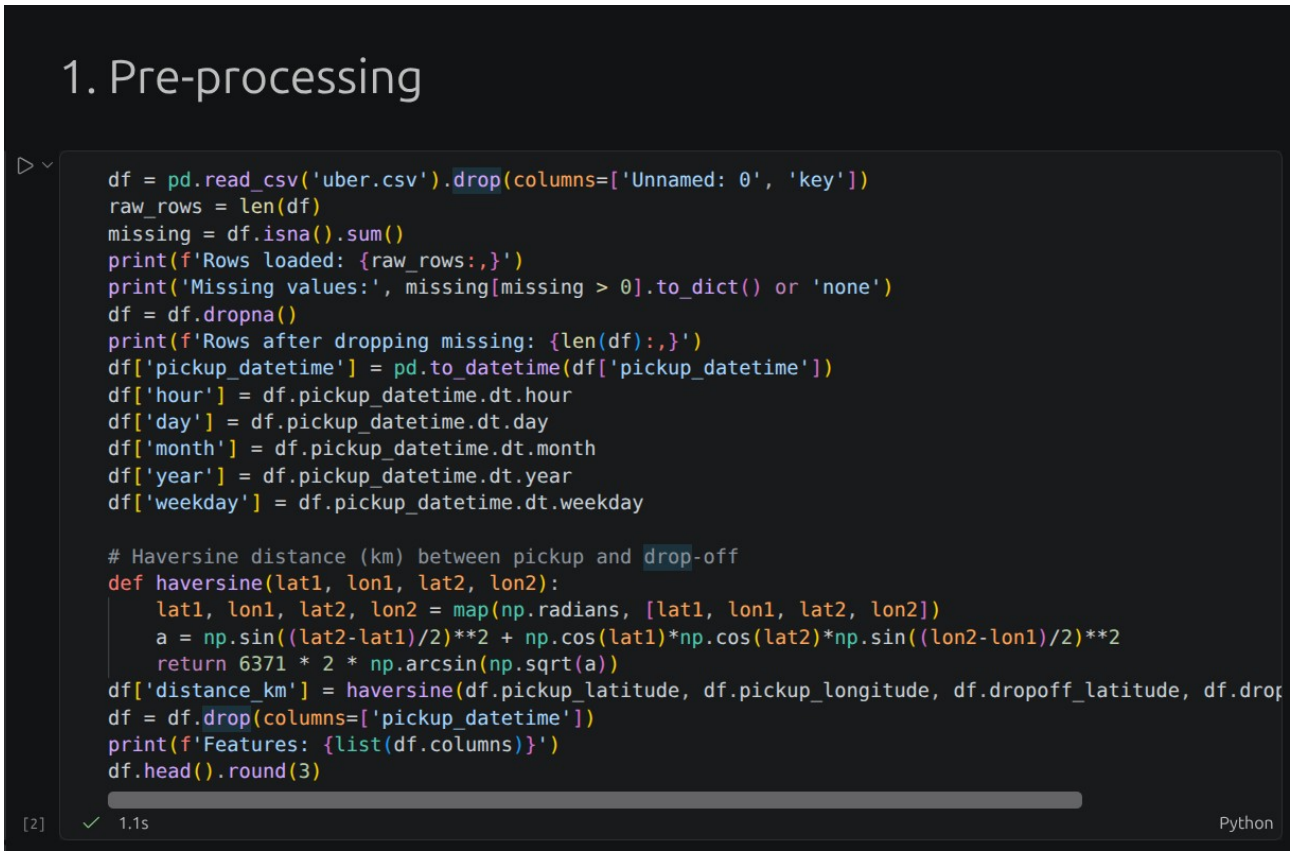
LP3 Assignment B1

Roll no: 41168

Problem Statement:
Predict the price of the Uber ride from a given pickup point to the agreed drop-off location.
Perform following tasks:
1. Pre-process the dataset.
2. Identify outliers.
3. Check the correlation.
4. Implement linear regression and random forest regression models.
Evaluate the models and compare their respective scores like R2, RMSE, etc.
Dataset link: Uber Fares Dataset (Kaggle)
https://www.kaggle.com/datasets/yasserh/uber-fares-dataset





```
Rows loaded: 200,000
Missing values: {'dropoff_longitude': 1, 'dropoff_latitude': 1}
Rows after dropping missing: 199,999
Features: ['fare_amount', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitud
```

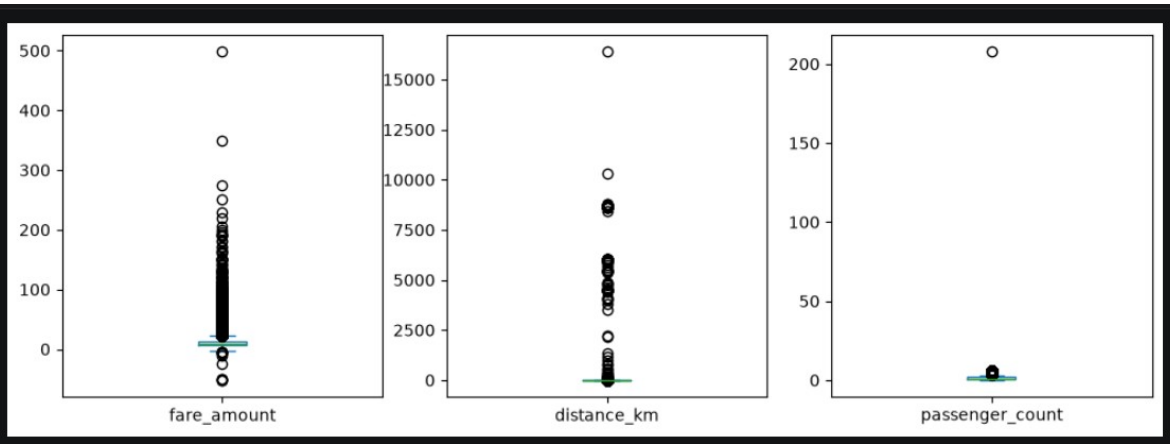
	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour	day
0	7.5	-74.000	40.738	-74.000	40.723	1	19	7
1	7.7	-73.994	40.728	-73.995	40.750	1	20	17
2	12.9	-74.005	40.741	-73.963	40.773	1	21	24
3	5.3	-73.976	40.791	-73.965	40.803	3	8	26
4	16.0	-73.925	40.744	-73.973	40.761	5	17	28

2. Outliers

```python
cols = ['fare_amount', 'distance_km', 'passenger_count']
df[cols].plot(kind='box', subplots=True, layout=(1, 3), figsize=(12, 4)); plt.show()

# Remove invalid values, then IQR-based outliers on fare and distance
n0 = len(df)
df = df[(df.fare_amount > 0) & (df.passenger_count.between(1, 6)) & (df.distance_km > 0)
        & df.pickup_latitude.between(-90, 90) & df.dropoff_latitude.between(-90, 90)
        & df.pickup_longitude.between(-180, 180) & df.dropoff_longitude.between(-180, 180)]
for c in ['fare_amount', 'distance_km']:
    q1, q3 = df[c].quantile([.25, .75]); iqr = q3 - q1
    df = df[df[c].between(q1 - 1.5*iqr, q3 + 1.5*iqr)]
print(f'Removed {n0 - len(df)} rows, {len(df)} remain')
```

[3] ✓ 0.2s Python



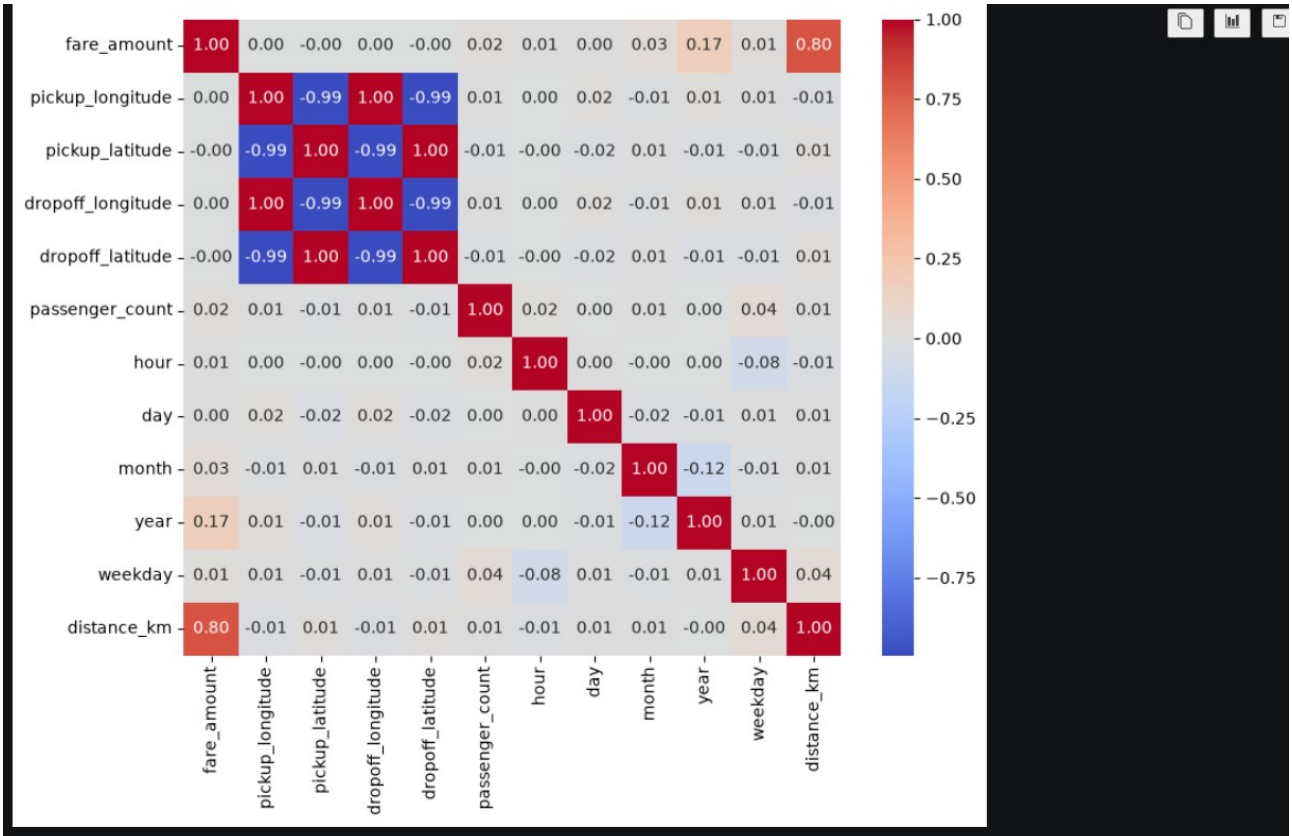

```
Removed 30980 rows, 169019 remain
```

3. Correlation

```
plt.figure(figsize=(9, 7))
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm'); plt.show()
corr = df.corr()['fare_amount'].drop('fare_amount')
print(f"Strongest correlation with fare_amount: {corr.abs().idxmax()} ({corr[corr.abs().idxmax()]:.
print(f"Weakest: {corr.abs().idxmin()} ({corr[corr.abs().idxmin()]:.2f})")

# Features with correlation 0.00 (rounded to 2 decimals): dropped for Linear Regression only
zero_corr = corr[corr.round(2) == 0].index.tolist()
print(f'Excluded from Linear Regression (correlation = 0.00): {zero_corr}')
```

[4] ✓ 0.3s Python




```
Strongest correlation with fare_amount: distance_km (0.80)
Weakest: pickup_longitude (0.00)
Excluded from Linear Regression (correlation = 0.00): ['pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitu
```

4. Models

Comparison

```
X = df.drop(columns='fare_amount'); y = df.fare_amount
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

models = {'Linear Regression': LinearRegression(),
          'Random Forest': RandomForestRegressor(n_estimators=100, n_jobs=-1, random_state=42)}
rows = []
for name, m in models.items():
    cols = X.columns.drop(zero_corr) if name == 'Linear Regression' else X.columns  # Random Forest
    m.fit(X_tr[cols], y_tr); p = m.predict(X_te[cols])
    rows.append({'Model': name, 'R2': r2_score(y_te, p),
                 'RMSE': np.sqrt(mean_squared_error(y_te, p)), 'MAE': mean_absolute_error(y_te, p)})
pd.DataFrame(rows).set_index('Model').round(4)
```

[5] ✓ 8.3s Python

Model	R2	RMSE	MAE
Linear Regression	0.6708	2.1339	1.5250
Random Forest	0.7482	1.8663	1.2985

(a) Linear Regression

```
lr = models['Linear Regression']
print('Intercept:', round(lr.intercept_, 4))
pd.Series(lr.coef_, index=X.columns.drop(zero_corr), name='Coefficient').round(4).sort_values()
```

[6] ✓ 0.0s Python

```
Intercept: -726.6746

weekday           -0.0326
hour               0.0086
passenger_count    0.0337
month              0.0524
year               0.3629
distance_km        2.0700
Name: Coefficient, dtype: float64
```

(b) Random Forest

```
rf = models['Random Forest']
pd.Series(rf.feature_importances_, index=X.columns, name='Importance').round(4).sort_values(ascendi
```

[7] ✓ 0.0s Python

```
distance_km          0.6765
year                 0.0517
dropoff_latitude     0.0483
dropoff_longitude    0.0436
pickup_longitude     0.0405
pickup_latitude      0.0363
hour                 0.0335
day                  0.0235
month                0.0211
weekday              0.0174
passenger_count      0.0078
Name: Importance, dtype: float64
```

Conclusion: the model with the higher R^2 and lower RMSE/MAE is the better predictor. Random Forest typically wins because it captures non-linear effects (e.g. time of day, year-based fare changes) that Linear Regression cannot.